

Mini-projet 15 — Timeouts & robustesse réseau

Pourquoi ce mini-projet ?

Ton scanner du MP14 répond « ouvert » ou « fermé ». Mais dans la vraie vie, il existe un **3e cas** : le port **filtré** (un pare-feu qui avale la connexion sans répondre → ça **traîne** jusqu'au timeout). Un vrai outil doit **distinguer** ces trois états, et ne jamais planter face à une cible capricieuse (hôte injoignable, nom invalide...). C'est ce qui sépare un jouet d'un outil fiable — et ça reprend directement la gestion d'erreurs du MP05.

1. Les trois états d'un port

État	Ce qui se passe sur le réseau	Côté Python
ouvert	un service accepte la connexion	<code>connect_ex</code> renvoie <code>0</code>
fermé	la machine refuse activement (RST)	<code>connect_ex</code> renvoie un code non-nul (ex. 111 Linux / 10061 Windows)
filtré	un pare-feu ignore le paquet (aucune réponse)	le timeout expire → exception <code>socket.timeout</code>

Le point clé, contre-intuitif : « fermé » et « filtré » ne se manifestent pas pareil.

- **Fermé** = réponse **rapide** (« non ! ») → un **code d'erreur** renvoyé par `connect_ex`.
- **Filtré** = **silence** → `connect_ex` ne renvoie rien, il **lève** `socket.timeout` quand le délai est dépassé.

Donc pour attraper « filtré », il faut un `try/except socket.timeout` autour de `connect_ex`.

2. Distinguer les trois

```
import socket

def etat_port(ip, port, timeout=1):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        try:
            code = s.connect_ex((ip, port))
        except socket.timeout:
            return "filtré" # silence = pare-feu qui drop
        return "ouvert" if code == 0 else "fermé"
```

Note : depuis Python 3.10, `socket.timeout` est un alias de `TimeoutError` — les deux marchent. On garde `socket.timeout`, lisible et compatible partout.

3. Ne jamais planter : les autres erreurs réseau

Deux autres pièges classiques quand la cible est « exotique » :

Situation	Exception levée
Nom d'hôte introuvable (DNS)	<code>socket.gaierror</code>
Réseau injoignable, route absente...	<code>OSError</code> (la classe parente de tout)

Un scanner robuste les attrape pour renvoyer un état propre au lieu de crasher :

```
except socket.timeout:
    return "filtré"
except socket.gaierror:
    return "hôte introuvable"
except OSError:
    return "erreur réseau" # filet de sécurité, EN DERNIER (du précis au général, MP05)
```

Tu réutilises le MP05 : l'ordre des `except` va du **plus précis** (`socket.timeout`, `socket.gaierror`) au **plus général** (`OSError`). Si tu mettais `OSError` en premier, il avalerait tout.

4. Bien choisir le timeout

Le timeout est un **compromis** :

- trop **court** (0.1 s) → tu rates des ports lents à répondre (faux « filtré »)
- trop **long** (10 s) → un seul port filtré fait traîner tout le scan

En pratique : **0.5 à 2 s** sur un réseau local (ton lab), plus sur Internet. Sur ta machine (`127.0.0.1`), tout est instantané, donc le timeout ne se déclenchera quasi jamais — c'est normal.

🎯 Ta mission

Objectif : faire évoluer ton scanner pour qu'il renvoie un **état** précis et ne plante jamais.

1. Écris `etat_port(ip, port, timeout=1)` qui renvoie une **chaîne** : `"ouvert"`, `"fermé"`, ou `"filtré"` (sur timeout).
2. Rends-la **robuste** : ajoute la gestion de `socket.gaierror` (→ `"hôte introuvable"`) et de `OSError` (→ `"erreur réseau"`), dans le bon ordre.

Tests (serveur `python -m http.server 8080` lancé) :

```
print(etat_port("127.0.0.1", 8080))    # -> ouvert
print(etat_port("127.0.0.1", 9999))    # -> fermé
print(etat_port("machin.invalid.xyz", 80)) # -> hôte introuvable
# "filtré" est dur à tester en local (localhost répond toujours vite) ;
# tu le verras sur une vraie cible protégée par pare-feu, dans ton lab.
```

Bonus : réintègre ton générateur `plage_ports` (MP11) et affiche l'état de chaque port d'une plage — mais n'affiche que les ports **ouverts**, comme un vrai scanner :

```
for p in plage_ports(8078, 8082):
    etat = etat_port("127.0.0.1", p)
    if etat == "ouvert":
        print(f"port {p} : {etat}")
# -> port 8080 : ouvert
```

✓ Critères de réussite

- `etat_port` renvoie `"ouvert"` / `"fermé"` correctement sur ta machine.
- Un nom d'hôte bidon renvoie `"hôte introuvable"` au lieu de crasher.
- Les `except` vont du plus précis au plus général.
- La socket est fermée (via `with`).

Si tu bloques — indices progressifs

Étape	Indice
Attraper le filtré	Mets <code>connect_ex</code> dans un <code>try</code> , et <code>except socket.timeout: return "filtré"</code> .
ouvert vs fermé	Après le <code>try</code> (si pas de timeout) : <code>return "ouvert" if code == 0 else "fermé"</code> .
Hôte introuvable	Ajoute <code>except socket.gaierror: return "hôte introuvable"</code> — c'est ce que lève la résolution DNS d'un nom bidon.
Ordre des except	<code>socket.timeout</code> et <code>socket.gaierror</code> d'abord (précis), <code>OSError</code> en dernier (général).

Où ranger : `mini-projets/15-timeouts-robustesse/scanner_robuste.py`.

Quand c'est prêt : `git add .` → `commit` → `push`, puis « c'est poussé » + ce que tes tests affichent.